

Informe de Entrega - Proyecto Final Programación I

Integrantes: Daniel Garibello, Rosa Maria Marin, Diana Atahualpa Zapata Villada, Diego Alejandro Gaviria Alzate

Asignatura: Programación I - Universidad de Manizales

Para este proyecto final, se solicitaba seleccionar un sistema de la información de la vida diaria y modelarlo aplicando los pilares esenciales del programacion orientado a objetos, enfatizando en la inicialización de estados y la interacción de componentes mediante constructores, instancias y métodos. Como equipo elegimos el desarrollo de un sistema de control de acceso perimetral con escáner biométrico para entornos empresariales. El objetivo del software es validar si los usuarios cuentan con los privilegios requeridos para ingresar a un área restringida y automatizar el bloqueo de seguridad en caso de detectar un intento de intrusión por parte de un usuario no registrado.

La arquitectura del código se organizó de forma modular, distribuyendo las responsabilidades en paquetes y directorios independientes. Esta separación de conceptos evita el acoplamiento de lógica en un único archivo, estructurando el proyecto de una manera limpia que facilita el desarrollo colaborativo y el control de versiones en Git.

En el módulo Empleado definimos la entidad del personal aplicando el concepto de encapsulamiento. Los atributos esenciales como el nombre, el identificador y el nivel de acceso se definieron con un alcance privado mediante el uso del doble guion bajo (`__nombre`, `__id_empleado`, `__nivel_acceso`). Con esto garantizamos que los datos sensibles queden protegidos contra modificaciones arbitrarias externas, forzando al resto del sistema a consultar dicha información únicamente a través de los métodos de lectura públicos o *getters* definidos en la clase.

Para el modelado del hardware de seguridad, implementamos una clase base denominada Dispositivo que centraliza las propiedades comunes de los equipos, como su nombre y ubicación geográfica dentro de la planta. A partir de esta clase aplicamos el principio de herencia para estructurar la clase ScannerBiometrico, la cual invoca al constructor de la clase padre mediante `super().__init__()` y añade un atributo especializado para la precisión mínima de lectura. Asimismo, este componente implementa un contrato formal a través de la interfaz abstracta IAutenticable, desarrollada con el módulo nativo abc para obligar a que el hardware defina la lógica concreta del método `autenticar()`.

Otro requerimiento clave del taller fue el uso de la composición. En lugar de sobrecargar la clase del escáner con algoritmos criptográficos, instanciamos un objeto independiente de la clase Encriptador directamente en el constructor de ScannerBiometrico. De este modo, al validar un acceso, el dispositivo delega la responsabilidad del enmascaramiento de datos a su

componente interno, protegiendo el ID de los empleados en los registros logs de auditoría sin que el escáner requiera conocer el funcionamiento interno del cifrado.

La gestión y evaluación perimetral se concentra en la clase Analizador. El método `procesar_intentos` hace uso del polimorfismo al recibir una lista genérica de dispositivos de seguridad, recorriendo la colección en un ciclo para ejecutar la validación independientemente del tipo de hardware específico que se esté evaluando. Dentro de este flujo se integró el manejo de excepciones de negocio. Si el analizador detecta que el nivel de acceso de un objeto empleado es inferior a 1, interrumpe la ejecución ordinaria del programa y eleva una excepción personalizada mediante la instrucción `raise AlertaSeguridadError`.

Para el archivo principal `main.py`, adaptamos el entorno de pruebas para reflejar un desarrollo grupal estructurado. Instanciamos cuatro objetos de la clase `Empleado` correspondientes a los integrantes del equipo (Daniel, Rosa María, Diana y Diego Alejandro), asignándoles credenciales válidas y niveles de acceso autorizados. Posteriormente, los agrupamos en una colección secuencial (lista) para procesar sus ingresos de manera dinámica mediante un ciclo `for`. Al final de la rutina, incluimos un objeto "Desconocido" con nivel de privilegios cero para verificar la efectividad de los mecanismos de contención ante amenazas.

Al ejecutar el programa, la consola arroja una salida secuencial clara. El sistema procesa e ilustra individualmente el éxito de la validación biométrica y el cifrado de datos para cada uno de los cuatro estudiantes autorizados del grupo. Inmediatamente después, al evaluar al usuario no identificado, el sistema intercepta la falta de privilegios, deniega el acceso físico y dispara de forma controlada el bloque `except`. La terminal muestra la captura de la excepción `AlertaSeguridadError`, imprimiendo la advertencia crítica del sistema de bloqueo y deteniendo el hilo de ejecución para proteger el perímetro.

Como conclusión, este laboratorio nos permitió comprender el impacto real de la programación orientada a objetos en el diseño de software robusto. La implementación de la herencia y el polimorfismo dota al sistema de una gran escalabilidad, permitiendo incorporar nuevos tipos de dispositivos lectores en el futuro sin necesidad de alterar la lógica central del analizador. Adicionalmente, el aislamiento de los datos sensibles y la gestión de contingencias mediante excepciones propias nos proporciona una visión clara de cómo se estructuran las aplicaciones bajo estándares profesionales de ingeniería.